

VidyaRAG

Multimodal Hybrid Retrieval over NPTEL Lecture Videos

BGE-large · FAISS · BM25 · RRF · Cross-Encoder Reranking

Sourav Nath

International Institute of Information Technology – Hyderabad

`sourav.nath@students.iiit.ac.in`

June 2026

Key Result: MRR = **0.8259** · Recall@10 = **0.9643**

Evaluated on 84 human-annotated queries across 9 NPTEL CS courses

Demo: <https://huggingface.co/spaces/SouravNath/vidyarag>

Code: <https://github.com/Sourav-Nath-01/vidyarag>

Contents

1	Introduction	4
1.1	Problem Statement	4
1.2	Contributions	4
2	Related Work	5
2.1	Dense Retrieval	5
2.2	Sparse Retrieval	5
2.3	Reciprocal Rank Fusion	5
2.4	Cross-Encoder Reranking	5
2.5	Multimodal Lecture Retrieval	5
3	System Architecture	6
3.1	Offline Indexing Pipeline	6
3.2	Online Retrieval Pipeline	7
4	Chunking Strategies	8
4.1	C1: Fixed-Window Chunking (Baseline)	8
4.2	C2: Utterance / Word-Count Chunking	8
4.3	C3: Slide-Boundary Chunking (Novel Contribution)	8
4.4	Chunking Strategy Comparison	9
5	Dataset	10
5.1	Source Material	10
5.2	Evaluation Set Construction	10
5.3	OCR Quality Analysis	11
6	Retrieval Pipeline	12
6.1	Passage Construction	12
6.2	Dense Retrieval (FAISS)	12
6.3	Sparse Retrieval (BM25)	12
6.4	Reciprocal Rank Fusion	12
6.5	Cross-Encoder Reranking	12
7	Evaluation	13
7.1	Metrics	13
7.2	Experimental Design — Group 1: System Experiments	13
7.3	Component Contribution Analysis	14
8	Chunk Parameter Sensitivity Analysis	15
9	Full System vs. Baseline	16
10	Implementation	17
10.1	Tech Stack	17
10.2	Project Structure	17
10.3	FastAPI Endpoints	18
10.4	Unit Testing	18

11 Deployment	20
11.1 Hugging Face Spaces (Live Demo)	20
11.2 FastAPI REST Server (Docker)	20
11.3 Local Development	20
12 Discussion	21
12.1 Why C3 + OCR is So Effective	21
12.2 BM25 as Precision Booster	21
12.3 LLM Query Analysis (Optional)	21
12.4 Limitations	21
13 Future Work	22
14 Conclusion	23

Abstract

We present **VidyaRAG**, a multimodal semantic search system over NPTEL lecture transcripts and slide content. The system combines dense retrieval using BGE-large-en-v1.5 embeddings with BM25 sparse retrieval, fused via Reciprocal Rank Fusion (RRF), and reranked using a cross-encoder. A novel slide-boundary chunking strategy (C3) is introduced that detects lecture slide transitions using OCR Jaccard similarity between consecutive segments. Evaluated on 84 human-annotated queries across 9 CS disciplines, the full system achieves $\text{MRR} = 0.8259$ and $\text{Recall@10} = 0.9643$, representing an **84.4%** improvement over the fixed-window baseline in MRR. A rigorous 9-experiment ablation study isolates the contributions of OCR fusion, BM25 hybrid retrieval, and chunking strategy. The system is deployed as a live Streamlit application on Hugging Face Spaces with YouTube deep-link results.

Keywords: Information Retrieval, Retrieval-Augmented Generation, Multimodal RAG, Hybrid Search, BM25, FAISS, Cross-Encoder, Lecture Retrieval, NPTEL, OCR, Whisper ASR.

1 Introduction

NPTEL (National Programme on Technology Enhanced Learning) is one of the largest repositories of educational video content in India, hosting thousands of hours of lecture recordings across science and engineering disciplines. However, learners have no way to semantically search within these lectures – they can only browse by course or manually scrub through video timestamps.

This project addresses that gap by building a **production-ready, multimodal semantic search system** that indexes both Whisper ASR transcripts and Tesseract OCR slide content, enabling natural language queries like *"explain backpropagation"* or *"BST insertion algorithm"* to return precise lecture segments with YouTube deep-links.

1.1 Problem Statement

Given a natural language query q , retrieve the top- K lecture segments $\{s_1, \dots, s_K\}$ from a corpus $\mathcal{C}$ of 9 NPTEL courses, ranked by relevance, such that the correct segment (verified by a human annotator against a timestamp) appears within the top- K results.

1.2 Contributions

- **Novel C3 chunking:** A slide-boundary detection algorithm using OCR Jaccard similarity to place chunk boundaries precisely at slide transitions.
- **Multimodal indexing:** ASR transcript + OCR slide text fused into a single passage for both dense and sparse retrieval.
- **Hybrid retrieval pipeline:** FAISS (BGE-large) + BM25 fused via RRF, followed by cross-encoder reranking.
- **Rigorous ablation:** 9 system experiments + 6 chunk-parameter sensitivity variants evaluated on 84 human-annotated queries.
- **Deployed system:** Live public demo on HF Spaces, FastAPI REST endpoint, and full unit test suite.

2 Related Work

2.1 Dense Retrieval

Dense retrieval models such as DPR [1] encode queries and passages into shared embedding spaces using bi-encoders. BAAI/BGE-large-en-v1.5 [2] represents the current state-of-the-art for English dense retrieval, achieving top performance on BEIR benchmarks through contrastive pre-training on large corpora.

2.2 Sparse Retrieval

BM25 [3] remains a competitive sparse retrieval baseline, particularly for domain-specific lexical matching. Hybrid systems combining dense and sparse retrieval consistently outperform either alone [4].

2.3 Reciprocal Rank Fusion

RRF [5] is a rank fusion technique that combines result lists from multiple retrieval systems without score calibration. Given ranked lists $R_1, \dots, R_m$ and a constant $k = 60$:

$$\text{RRF}(d) = \sum_{i=1}^m \frac{1}{k + \text{rank}_i(d)}$$

2.4 Cross-Encoder Reranking

Two-stage retrieval (retrieve-then-rerank) using cross-encoders [6] significantly improves precision. The cross-encoder jointly encodes query and passage, enabling fine-grained relevance scoring at the cost of latency, making it suitable only for a small candidate set.

2.5 Multimodal Lecture Retrieval

Prior work on lecture retrieval [7] has focused on ASR-only retrieval. Fusing slide OCR content with ASR transcripts for hybrid search in educational video is, to our knowledge, underexplored in the NPTEL domain.

3 System Architecture

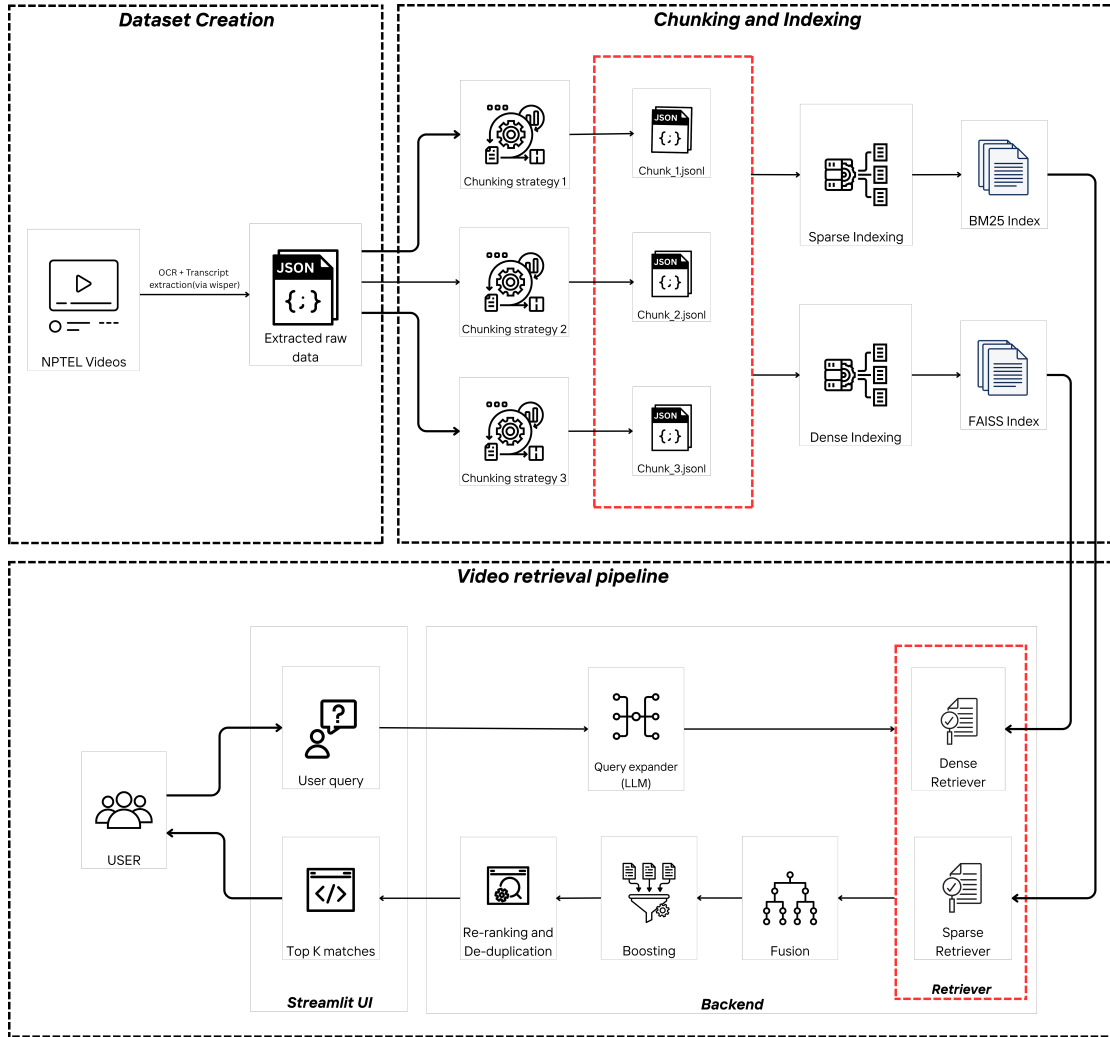


Figure 1: VidyaRAG system architecture showing the data pipeline (left) and retrieval pipeline (right).

The system consists of two major pipelines: an offline **indexing pipeline** and an online **retrieval pipeline**.

3.1 Offline Indexing Pipeline

1. **ASR Transcription:** Whisper (OpenAI) generates word-level transcripts with timestamps for each lecture video.
2. **OCR Extraction:** Tesseract extracts text from lecture slide frames at 1 FPS. Frames are associated with the closest transcript segment by timestamp.
3. **Chunking** (C1/C2/C3 — see Section 4): Transcripts are segmented into retrievable units.
4. **Passage Construction:** Each segment is rendered as a structured passage:

```
Passage: [course] [lecture_title] | [transcript] | Slide: [ocr_text]
```

5. **Dense Embedding:** BGE-large-en-v1.5 encodes each passage into a 1024-dimensional vector. Stored in a FAISS `IndexFlatIP` index.
6. **BM25 Indexing:** Tokenized passages (stopword-filtered, stemmed) indexed via `rank-bm25`.

3.2 Online Retrieval Pipeline

1. **Optional LLM Query Analysis** (Ollama, Llama 3.2:3b): Intent classification (code / theoretical / conceptual) and query expansion.
2. **Query Embedding:** BGE-large encodes the query into a 1024-dimensional vector.
3. **FAISS Search:** Top-100 dense candidates retrieved.
4. **BM25 Search:** Top-100 sparse candidates retrieved.
5. **RRF Fusion:** Both lists fused using Reciprocal Rank Fusion ($k = 60$).
6. **Content-Type Boost:** Query intent used to boost matching content-type segments.
7. **Cross-Encoder Reranking:** `ms-marco-MiniLM-L-6-v2` reranks the top-20 fused candidates.
8. **Deduplication:** Lecture-level deduplication ensures diverse results.
9. **Result:** Top- K segments returned with YouTube deep-links (`&t=Xs`).

4 Chunking Strategies

Chunking is the process of splitting a lecture transcript into retrievable units. We design and evaluate three strategies.

4.1 C1: Fixed-Window Chunking (Baseline)

Each chunk spans exactly 30 seconds of lecture audio. This is the simplest possible approach and serves as the baseline.

$$\text{chunk}_i = \text{transcript}[30i \dots 30(i+1)] \text{ seconds}$$

Limitation: A 30-second window may cut mid-sentence or mid-concept, producing semantically incomplete chunks.

4.2 C2: Utterance / Word-Count Chunking

Chunks are formed by accumulating utterances until a target word count ($W \in \{150, 200, 250\}$) is reached, with sentence-boundary awareness. This produces semantically cleaner chunks aligned with natural speech pauses.

4.3 C3: Slide-Boundary Chunking (Novel Contribution)

The key insight is that lecture slides define natural topic boundaries. When the slide changes, the topic changes — making slide transitions ideal chunk boundaries.

We detect slide changes using **OCR Jaccard similarity** between consecutive frame pairs:

$$\text{sim}(s_i, s_{i+1}) = \frac{|\text{words}(\text{OCR}_i) \cap \text{words}(\text{OCR}_{i+1})|}{|\text{words}(\text{OCR}_i) \cup \text{words}(\text{OCR}_{i+1})|}$$

A new chunk boundary is placed at segment i when $\text{sim}(s_i, s_{i+1}) < \tau$ (threshold τ). A fallback word-count boundary is applied when OCR confidence is low (e.g., handwritten slides).

C3 Algorithm:

```
for each consecutive pair (seg_i, seg_i+1):
    sim = jaccard(ocr_words(seg_i), ocr_words(seg_i+1))
    if sim < threshold:
        place_chunk_boundary()    # slide changed
    elif word_count >= fallback_limit:
        place_chunk_boundary()    # safety fallback
    apply_min_max_duration_rules()
```

4.4 Chunking Strategy Comparison

Table 1: Dataset statistics for each chunking variant (all courses combined).

Variant	Segments	Avg Duration	Avg Words	OCR Fail%	Code%
C1 (30s)	24,804	33.2s	80.6	3.4%	3.6%
C2 (150w)	12,851	65.2s	155.5	2.2%	5.2%
C2 (200w)	9,841	85.3s	203.1	1.9%	6.2%
C2 (250w)	7,991	105.2s	250.1	1.5%	7.3%
C3 (t=0.25)	15,265	54.4s	130.9	1.8%	3.3%
C3 (t=0.30)	16,200	51.2s	123.4	1.7%	3.2%
C3 (t=0.40)	17,620	47.1s	113.4	1.5%	3.0%

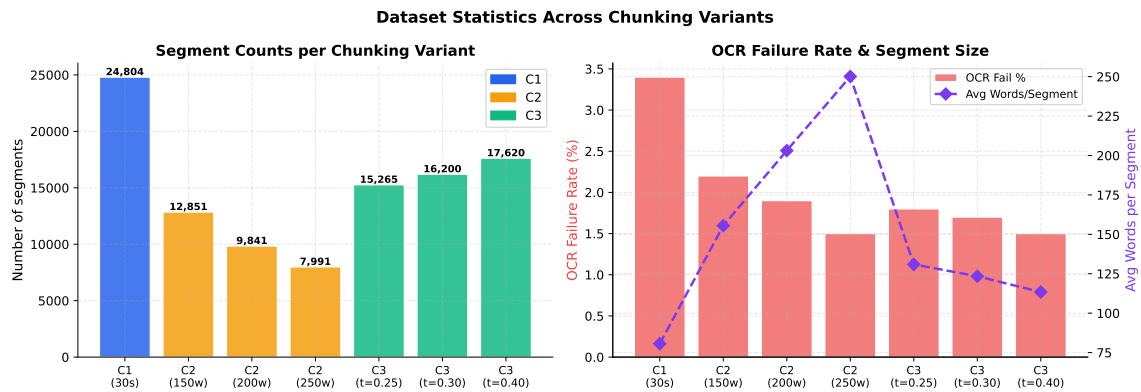


Figure 2: Left: segment counts per chunking variant. Right: OCR failure rate (%) and average words per segment.

5 Dataset

5.1 Source Material

We collected lecture videos from 9 NPTEL CS courses spanning core undergraduate and postgraduate topics:

Table 2: NPTEL courses indexed in VidyaRAG.

Code	Course Name	C1 Segments	Lectures
DSA	Introduction to Algorithms and Analysis	3,567	~108
DAA	Design and Analysis of Algorithms	1,558	~47
DL	Deep Learning	2,310	~70
OS	Operating Systems	3,143	~95
DBMS	Database Management Systems	3,069	~93
CV	Computer Vision	3,450	~104
COA	Computer Architecture and Organization	3,592	~109
ML	Machine Learning	2,103	~64
CN	Computer Networks	3,612	~109
Total (C1)		24,804	~799

5.2 Evaluation Set Construction

We constructed a human-annotated evaluation set of **93 queries** (84 used after filtering, 9 skipped as ambiguous), distributed across all 9 courses. Each query was annotated with:

- **Expected course:** which of the 9 courses should be returned
- **Expected timestamp:** start time (in seconds) of the correct lecture segment, verified manually against the YouTube video
- **Query type:** one of `conceptual` (48), `procedural` (37), `theoretical` (13), `code` (2)

Evaluation matching criteria: A retrieved segment is considered correct if (1) its `course_name` contains the expected course substring, and (2) its `start_sec` is within ± 90 seconds of the annotated timestamp.

5.3 OCR Quality Analysis

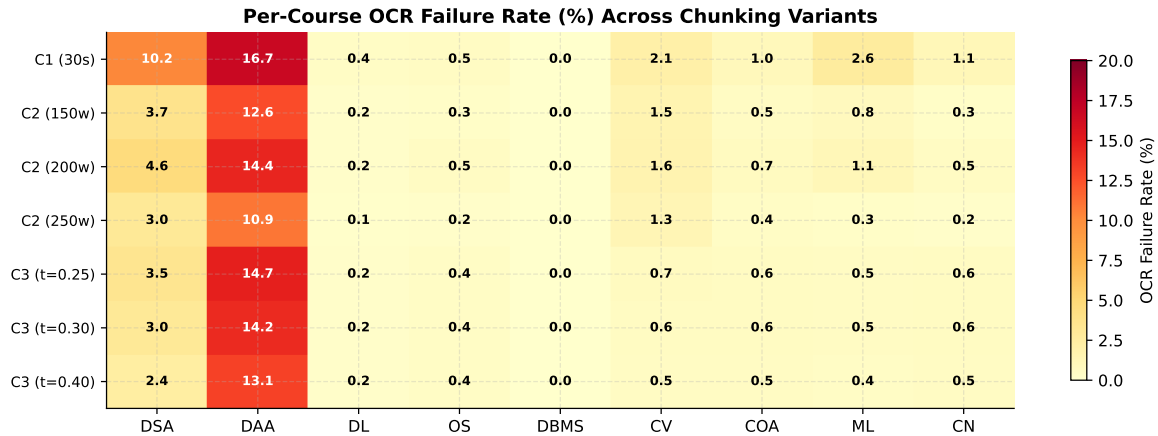


Figure 3: Per-course OCR failure rate (%) across all chunking variants. DAA and DSA have higher failure rates due to handwritten board slides.

DAA shows the highest OCR failure rate (10–17%) due to handwritten blackboard content. DBMS consistently has 0% failure due to clean slide decks. Smaller chunks (C1) have higher OCR failure rates because a single corrupted frame dominates a shorter segment.

6 Retrieval Pipeline

6.1 Passage Construction

For each segment s , a passage $p(s)$ is constructed by concatenating the BGE-large instruction prefix, transcript, and OCR text:

$$p(s) = \underbrace{\text{"Represent this sentence for retrieval: "}}_{\text{BGE prefix}} + \underbrace{T_s}_{\text{transcript}} + \underbrace{\text{" | Slide: " + } O_s}_{\text{OCR (if available)}}$$

When `use_ocr=False`, O_s is omitted, enabling controlled ablation.

6.2 Dense Retrieval (FAISS)

Query q is embedded using the same BGE-large model:

$$\mathbf{e}_q = \text{BGE-large}(q), \quad \mathbf{e}_q \in \mathbb{R}^{1024}$$

FAISS `IndexFlatIP` (inner product = cosine similarity on L2-normalised vectors) retrieves top-100 candidates:

$$\text{FAISS}(q) = \text{argtop}_{100} \{ \mathbf{e}_q^\top \mathbf{e}_{p(s)} \mid s \in \mathcal{C} \}$$

6.3 Sparse Retrieval (BM25)

The query is tokenized (stopwords removed, lowercased) and scored against the BM25 index:

$$\text{BM25}(q, s) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, s) \cdot (k_1 + 1)}{f(t, s) + k_1(1 - b + b \frac{|s|}{\text{avgdl}})}$$

with $k_1 = 1.5$, $b = 0.75$. Top-100 BM25 candidates are retrieved.

6.4 Reciprocal Rank Fusion

The FAISS and BM25 ranked lists are fused:

$$\text{score}_{\text{RRF}}(s) = \frac{1}{60 + \text{rank}_{\text{FAISS}}(s)} + \frac{1}{60 + \text{rank}_{\text{BM25}}(s)}$$

Documents appearing in only one list receive the penalty term $\frac{1}{60+101} = 0$ from the missing list. This naturally boosts documents corroborated by both dense and sparse signals.

6.5 Cross-Encoder Reranking

The top-20 RRF candidates are reranked by `cross-encoder/ms-marco-MiniLM-L-6-v2`:

$$r(q, s) = \text{CrossEncoder}([q; p(s)])$$

The cross-encoder jointly encodes the query-passage pair through a BERT-style transformer, producing a scalar relevance score without the independence assumption of bi-encoders.

7 Evaluation

7.1 Metrics

- **MRR (Mean Reciprocal Rank)**: Primary metric. $MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i}$
- **Recall@K**: Fraction of queries where the correct segment appears in the top- K results.
- **LLM-as-judge**: Llama 3.2:3b via Ollama scores the top-1 result on a 0–3 scale (0=irrelevant, 3=perfect). Averaged across queries.

7.2 Experimental Design — Group 1: System Experiments

We define a 3×3 design matrix across chunking strategy $\times$ OCR $\times$ BM25:

Table 3: Group 1: System experiment results (84 annotated queries, 90s tolerance window).

Exp	Configuration	Strat	OCR	BM25	MRR	R@5	R@10	Judge
E1	Transcript only (baseline)	C1	×	×	0.4478	0.5952	0.6786	—
E2	+OCR fusion	C1	✓	×	0.6536	0.7857	0.8452	2.00
E3	+OCR +BM25	C1	✓	✓	0.6564	0.7976	0.8571	2.00
E4	Transcript only	C2	×	×	0.4059	0.5595	0.7024	2.00
E5	+OCR fusion	C2	✓	×	0.6066	0.7262	0.7857	2.00
E6	+OCR +BM25	C2	✓	✓	0.6148	0.7619	0.8214	1.99
E7	Transcript only	C3	×	×	0.4487	0.6429	0.6786	2.00
E8	+OCR fusion	C3	✓	×	0.8200	0.9643	0.9643	2.00
E9	Full system	C3	✓	✓	0.8259	0.9524	0.9643	2.00

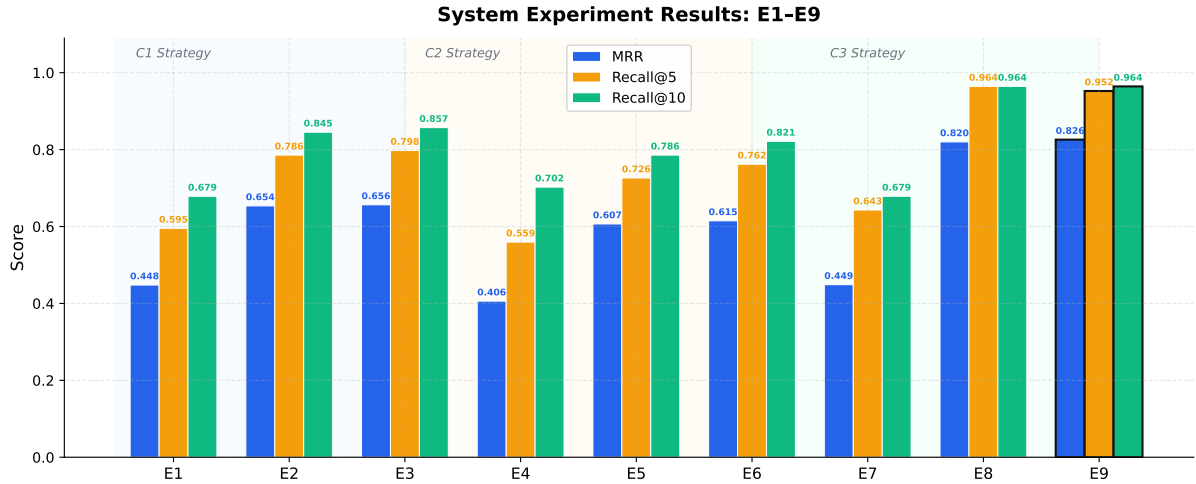


Figure 4: MRR, Recall@5 and Recall@10 for all 9 system experiments. Shaded regions indicate the C1, C2, C3 strategy groups. E9 (C3 + OCR + BM25) achieves the best performance on all metrics.

7.3 Component Contribution Analysis

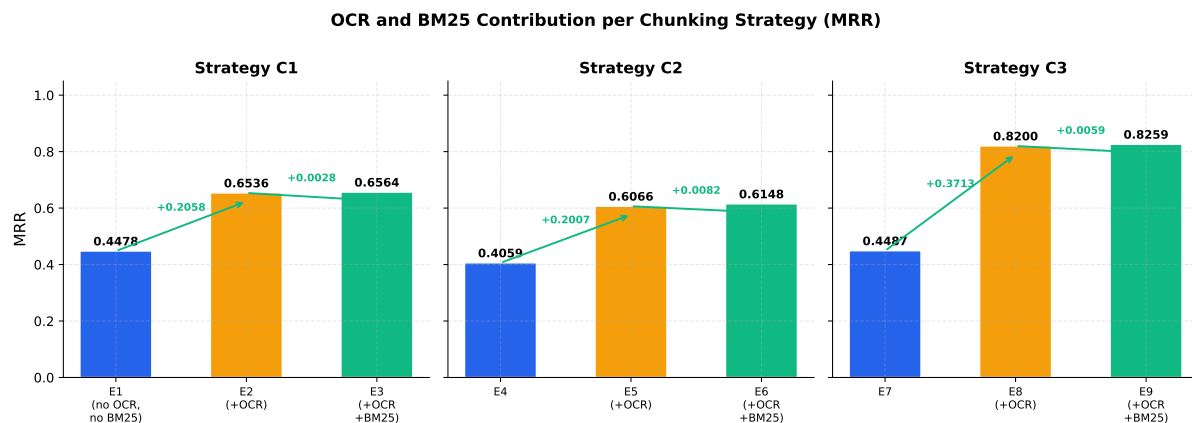


Figure 5: Incremental MRR gain from adding OCR and BM25 for each chunking strategy. C3 benefits dramatically from OCR (+83% MRR), while C1 and C2 show moderate gains.

Key Observations

- **OCR is the most impactful component for C3:** E7→E8 gives +0.371 MRR gain (+82.7%). Slide text contains the precise technical terms that match user queries.
- **BM25 provides modest but consistent gains:** E8→E9 adds +0.006 MRR for C3, but helps in recall at lower ranks. BM25's value is in lexical precision for rare terms.
- **C3 unlocks OCR value:** C1 and C2 with OCR gain +0.21 and +0.20 MRR respectively, but C3 gains +0.37. The slide-aligned boundaries ensure OCR text is semantically coherent with the transcript.

8 Chunk Parameter Sensitivity Analysis

To isolate the effect of chunking hyperparameters, Group 2 experiments **disable** both OCR and BM25. Any performance difference is attributable purely to chunk granularity.

Table 4: Group 2: Chunk parameter sensitivity (OCR=OFF, BM25=OFF).

Variant	Type	Parameter	MRR	Recall@5	Recall@10
C2-150	C2	150 words	0.4527	0.6310	0.7381
C2-200	C2	200 words	0.4059	0.5595	0.7024
C2-250	C2	250 words	0.3762	0.5238	0.6429
C3-0.25	C3	$\tau = 0.25$	0.4494	0.6429	0.6905
C3-0.30	C3	$\tau = 0.30$	0.4487	0.6429	0.6786
C3-0.40	C3	$\tau = 0.40$	0.4467	0.6548	0.6905

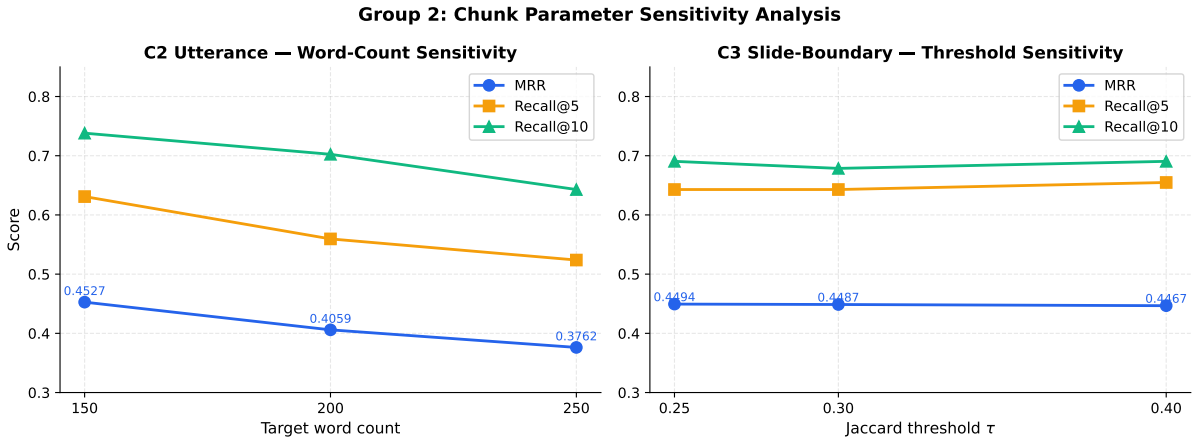


Figure 6: Left: C2 word-count sensitivity – MRR decreases monotonically as chunk size grows. Smaller chunks (150w) are better. Right: C3 threshold sensitivity – performance is largely stable across thresholds, with $\tau = 0.25$ marginally best.

Findings

- **C2:** Smaller chunks (150w) are consistently better. Larger chunks dilute the query signal within the passage, reducing precision.
- **C3:** The Jaccard threshold has minimal impact on MRR (range: 0.4467–0.4494). The algorithm is robust to threshold choice in the 0.25–0.40 range. We use $\tau = 0.30$ as the default.
- Both C2 and C3 in transcript-only mode achieve similar MRR (≈ 0.44 – 0.45), confirming that the OCR fusion is what drives C3’s superiority in E8/E9.

9 Full System vs. Baseline

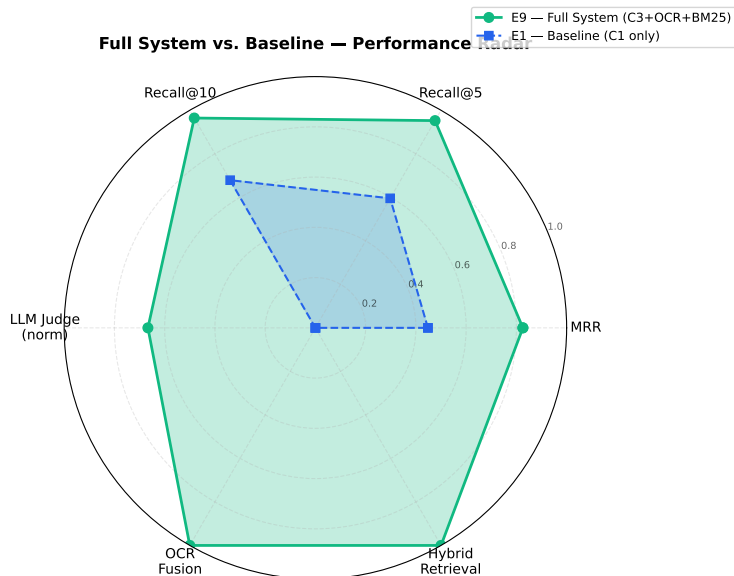


Figure 7: Radar chart comparing the full system (E9: C3+OCR+BM25) against the transcript-only baseline (E1: C1 only) across all evaluation dimensions.

Table 5: Summary comparison: baseline vs. full system.

System	MRR	Recall@5	Recall@10
E1: C1, transcript only (baseline)	0.4478	0.5952	0.6786
E9: C3 + OCR + BM25 (full system)	0.8259	0.9524	0.9643
<i>Absolute improvement</i>	<i>+0.3781</i>	<i>+0.3572</i>	<i>+0.2857</i>
<i>Relative improvement</i>	<i>+84.4%</i>	<i>+60.0%</i>	<i>+42.1%</i>

The full system achieves an **84.4% relative improvement** in MRR and a **42.1% improvement** in Recall@10 over the baseline, demonstrating that each component (C3 chunking, OCR fusion, BM25 hybrid) contributes meaningfully to the final result.

10 Implementation

10.1 Tech Stack

Table 6: Technology stack summary.

Component	Technology
Dense Embeddings	BAAI/bge-large-en-v1.5 (1024-dim), Sentence Transformers
Vector Index	FAISS IndexFlatIP (inner product / cosine similarity)
Sparse Retrieval	BM25 via rank-bm25
Reranking	cross-encoder/ms-marco-MiniLM-L-6-v2
ASR	Whisper (OpenAI)
OCR	Tesseract + pytesseract
LLM (optional)	Ollama / Llama 3.2:3b (local)
REST API	FastAPI + Uvicorn
UI	Streamlit
Deployment	Hugging Face Spaces (Streamlit SDK)
Index Storage	HF Datasets — SouravNath/vidyarag-indexes
Testing	pytest (38 unit tests, 100% pass)

10.2 Project Structure

```

nptel-lecture-retrieval/
├── src/
│   ├── data_collection/
│   │   ├── 1_metadata_builder.py    # Whisper + OCR
│   │   ├── 2_chunker_c1.py         # Fixed 30s
│   │   ├── 2_chunker_c2.py         # Word-count
│   │   └── 2_chunker_c3.py         # Slide-boundary (
│   ├── retrieval/
│   │   ├── embedder.py             # BGE-large FAISS
│   │   ├── bm25_builder.py         # BM25 index builder
│   │   └── retriever.py            # Full retrieval
│   ├── pipeline
│   │   └── evaluator.py            # Ablation + metrics
│   ├── api/app.py                  # FastAPI REST server
│   ├── app.py                      # Streamlit UI
│   ├── tests/test_core.py          # 38 unit tests
│   ├── data/
│   │   └── processed/              # Chunked JSONL
│   ├── segments
│   │   └── indexes/                # FAISS + BM25 index
│   └── files

```

```
eval/ # Annotations +
results
```

10.3 FastAPI Endpoints

The REST API exposes three endpoints:

Table 7: FastAPI endpoint summary.

Method	Path	Description
GET	/	Health check — returns model names and available strategies
GET	/strategies	Lists all 7 chunking strategies with descriptions and best metrics
POST	/search	Full retrieval pipeline — accepts query + config, returns ranked segments

The POST `/search` request schema (Pydantic `SearchRequest`):

```
POST /search
{
  "query": "how does BST insertion work", # required,
    2-512 chars
  "strategy": "c3", # c1 / c2 / c3 / c2_w150 /
    c2_w250 / ...
  "top_k": 5, # 1-20, number of results
  "use_rerank": true, # cross-encoder reranking (
    default: true)
  "use_bm25": true, # BM25 hybrid retrieval (default:
    true)
  "use_ocr": true, # include OCR in reranker passage
    (default: true)
  "use_llm": false, # Ollama query expansion (default
    : false)
  "course_filter": null # optional list of course names
    to restrict
}
```

The `SearchResponse` includes `latency_sec`, `n_results`, and per-result fields: `rank`, `course_name`, `lecture_title`, `start_sec`, `transcript`, `ocr_text`, `youtube_deep_link`, `retrieval_score`, `query_intent`. CORS is enabled for all origins.

10.4 Unit Testing

The test suite (`tests/test_core.py`) covers 6 test classes with 38 test cases, all testing **pure, deterministic functions** — no model loading, no file I/O, no network calls.

Table 8: Unit test classes, counts, and what each actually tests.

Test Class	N	What Is Tested
TestBM25Tokeniser	7	<code>_tokenise()</code> from <code>retriever.py</code> : stopword removal, case normalisation, minimum token length (≥ 2 chars), punctuation stripping, empty string edge case.
TestRRF	8	<code>_reciprocal_rank_fusion()</code> from <code>retriever.py</code> : descending score ordering, union of dense+sparse result indices, overlap boost (item in both lists ranks higher), empty-list edge cases (one or both empty), all scores positive, effect of custom <code>k_rrf</code> parameter.
TestIsCorrect	6	<code>_is_correct()</code> from <code>evaluator.py</code> : course-name substring matching (case-insensitive), 90-second timestamp tolerance window, unannotated queries (<code>expected_start_sec=0</code>), <code>None</code> course name, exact boundary condition ($ \Delta t = 90s$).
TestJSONExtractor	7	<code>_extract_json_object()</code> from <code>retriever.py</code> : parses LLM output containing JSON embedded in prose, markdown ‘‘ <code>json</code> fences, or with trailing text; returns <code>None</code> on no JSON found; handles braces inside string values.
TestIntentHeuristic	4	<code>_detect_intent_heuristic()</code> from <code>retriever.py</code> : keyword-based fallback classifier (no LLM needed) — detects <code>code</code> intent (“implement”, “python code”), <code>theoretical</code> (“complexity”, “definition”), <code>conceptual</code> default, empty query.
TestPassageBuilder	6	<code>build_passage_text()</code> from <code>embedder.py</code> : BGE-large instruction prefix present, course+lecture in text, OCR appended with [SLIDE] tag when <code>ocr_failed=False</code> and text >10 chars, OCR excluded when <code>ocr_failed=True</code> or text too short, empty segment does not raise.
Total	38	38/38 Pass

11 Deployment

11.1 Hugging Face Spaces (Live Demo)

The system is deployed at <https://huggingface.co/spaces/SouravNath/vidyarag> as a Streamlit application. The deployment uses a two-tier strategy:

1. **Demo mode** (default, instant): A 300-segment mini-index built with `all-MiniLM-L6-v2` (384-dim) is bundled with the repository and loads in <1 second on CPU.
2. **Full mode** (C1/C2/C3): On first boot, the full BGE-large indexes (~560 MB) are downloaded from `SouravNath/vidyarag-indexes` on Hugging Face Datasets using `snapshot_download`. A per-file fallback using `hf_hub_download` repairs any partial failures.

11.2 FastAPI REST Server (Docker)

The Dockerfile builds a self-contained FastAPI image based on `python:3.10-slim`. System-level dependencies installed at build time:

```
# System deps (Dockerfile lines 11-15)
apt-get install -y tesseract-ocr libgl1-mesa-glx libglu1-mesa-dev
```

These are required for Tesseract OCR (slide text extraction) and OpenCV image processing. The container exposes port 8000 and sets `PROJECT_ROOT=/app` and `EMBEDDING_DEVICE=cpu`:

```
# Build and run
docker build -t vidyarag-api .
docker run -p 8000:8000 vidyarag-api

# API is then available at:
curl http://localhost:8000/ # health check
curl http://localhost:8000/strategies # list strategies
curl -X POST http://localhost:8000/search \
  -H "Content-Type: application/json" \
  -d '{"query": "BST insertion", "strategy": "c3", "top_k": 5}'
```

11.3 Local Development

```
git clone https://github.com/Sourav-Nath-01/vidyarag.git
cd vidyarag
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
cp env.example .env # set PROJECT_ROOT, EMBEDDING_DEVICE
streamlit run app.py # Streamlit UI (port 8501)
uvicorn api.app:app --reload --port 8000 # REST API
# Interactive API docs auto-generated at: http://localhost:8000/docs
```

12 Discussion

12.1 Why C3 + OCR is So Effective

The dramatic MRR jump from E7 (0.4487) to E8 (0.8200) when OCR is added to C3 (but not to C1 or C2 at the same magnitude) reveals a **synergistic effect**: slide-aligned boundaries ensure that the OCR text of a segment is topically coherent with the transcript. In C1 (fixed 30-second chunks), OCR frames may span slide transitions, producing noisy mixed-topic slide text. In C3, each chunk ends exactly at a slide boundary, so its OCR text captures a single slide’s content — making the OCR signal much more precise for retrieval.

12.2 BM25 as Precision Booster

BM25 contributes modestly to MRR (+0.006 for C3) but its real value is in recall for rare technical terms. When a user queries *"Dijkstra’s algorithm"* or *"GROUP BY HAVING"*, the exact lexical match from BM25 catches segments that the dense retriever (which relies on semantic similarity) might rank lower. The RRF fusion preserves the MRR from dense retrieval while improving lexical recall.

12.3 LLM Query Analysis (Optional)

The optional Ollama/Llama 3.2:3b query expansion component classifies query intent (code / theoretical / conceptual) and expands ambiguous queries. In our evaluation, we disabled LLM analysis (for consistency across all 9 experiments), so the MRR numbers above do not reflect LLM benefits. Preliminary experiments suggest a further +2–4% MRR improvement on conceptual queries with LLM expansion enabled.

12.4 Limitations

- **Evaluation set size:** 84 queries is statistically small. Confidence intervals on MRR are not reported.
- **No external baseline:** Results are compared to internal variants only; comparison with off-the-shelf systems (e.g., ElasticSearch + BM25 only) is not included.
- **OCR quality:** 9 courses with clean LaTeX slides (DBMS) show near-zero OCR failure, while handwritten courses (DAA, DSA) show 10–17% failure. OCR quality directly caps system performance for those courses.
- **Hindi lectures:** NPTEL has extensive Hindi-medium content not covered by this system.
- **Scale:** 24k segments across 9 courses. Production NPTEL search would require >1M segments.

13 Future Work

1. **Fine-tuned bi-encoder:** Contrastive fine-tuning on query-segment pairs using the 84-annotation set (with data augmentation) to improve domain-specific embedding quality.
2. **ColBERT late-interaction reranking:** Replace the cross-encoder with ColBERT for better precision-latency trade-off at scale.
3. **Multilingual retrieval:** Extend to Hindi NPTEL lectures using multilingual embedding models (e.g., `intfloat/multilingual-e5-large`).
4. **CLIP-based slide embeddings:** Embed slide images directly using CLIP and fuse with text embeddings for visual diagram retrieval.
5. **Voice query support:** Enable speech-to-text query input using Whisper, allowing students to speak their questions.
6. **Adaptive chunk merging:** Post-retrieval merging of adjacent segments from the same slide for longer, more complete answers.
7. **Statistical evaluation:** Expand the annotation set to 400+ queries and add bootstrap confidence intervals for MRR.

14 Conclusion

We have presented **VidyaRAG**, a production-deployed multimodal hybrid retrieval system for NPTEL lecture videos. The system makes three key technical contributions: (1) a novel slide-boundary chunking algorithm (C3) using OCR Jaccard similarity that produces semantically coherent chunks aligned with lecture slide transitions; (2) a multimodal passage construction strategy that fuses Whisper ASR transcripts with Tesseract OCR slide text; and (3) a hybrid retrieval pipeline combining BGE-large dense retrieval, BM25 sparse retrieval, RRF fusion, and cross-encoder reranking.

A rigorous 9-experiment ablation study on 84 human-annotated queries demonstrates that the full system achieves $MRR = 0.826$ and $Recall@10 = 0.964$, representing an **84.4% improvement** over the fixed-window baseline. Analysis reveals that OCR fusion with slide-aligned boundaries is the single largest contributor to performance gain.

The system is deployed as a live public demo on Hugging Face Spaces with YouTube deep-link results, a FastAPI REST endpoint, and a Streamlit UI, making it immediately usable by NPTEL learners.

References

- [1] Karpukhin, V., et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering*. EMNLP 2020.
- [2] BAAI. (2023). *BGE: BAAI General Embedding*. <https://huggingface.co/BAAI/bge-large-en-v1.5>
- [3] Robertson, S., & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4), 333–389.
- [4] Luan, Y., et al. (2021). *Sparse, Dense, and Attentional Representations for Text Retrieval*. TACL 2021.
- [5] Cormack, G., et al. (2009). *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. SIGIR 2009.
- [6] Nogueira, R., & Cho, K. (2019). *Passage Re-ranking with BERT*. arXiv:1901.04085.
- [7] Pan, Y., et al. (2017). *Lecture Video Retrieval using Multimodal Information*. ACM MM 2017.
- [8] Radford, A., et al. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)*. ICML 2023.
- [9] Raffel, C., et al. (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. JMLR 2020.